Platform Timing Contracts: A Language and Instrumentation for Capturing SoC Timing Channels

***Abstract*—Variations in the time required by a computer to perform computations can leak confidential information. Constant-time programming techniques defend against such timing attacks, but make assumptions on the behavior of the underlying hardware. In response, previous work has introduced several techniques for verifying CPU compliance with hardware-software contracts. However, selecting which hardware-software contract to verify is non-trivial and requires careful consideration of the system's architecture and the specific timing attacks it may face. If a CPU is integrated into a platform that creates more timing channels than expected by a contract, CPU verification against this contract may be insufficient to guarantee security. In this paper, we address the integration of a verified constant-time CPU into a larger system. We make three contributions: (1) we define platform timing contracts as a language to express expected platform-induced timing channels, (2) we design a low-overhead synthesizable instrumentation that materializes these contracts and is compatible with the existing CPU verification methods, and (3) we demonstrate that this instrumentation can detect bugs that some existing CPU verification methods would miss when the CPU is integrated into a larger system.**

I. Introduction

Timing side-channel attacks exploit variations in execution time to leak secrets through measurable side effects. By carefully choreographing benign-looking instruction sequences, an attacker can steer microarchitectural state, such as caches, TLBs, execution ports, or branch predictors to make secret-dependent effects externally observable. Two broad families have emerged: transient and speculative execution attacks [1]–[6], which leave secret-dependent footprints during mis-speculated execution, and contention-driven channels [7]–[12], which measure interference to encode information. These attacks can often be mounted entirely in software, without special privileges or hardware access. Mitigation is notoriously difficult after fabrication. Software or microcode defenses are often partial [13] and impose significant overheads [14]. To address these challenges, recent work introduced hardware-software contracts, which inform developers about the side effects of their code on a CPU [15], [16]. As a result, several techniques have been proposed to formally verify that a CPU implementation complies with such contracts [17]–[22] to prevent unexpected confidentiality breaches before fabrication.

Platform timing channels. CPUs are often verified in isolation [17]–[22] against hardware-software contracts before integration into a larger system, both for scalability and because the exact RTL implementations of the integrating platforms are usually unknown during CPU design and verification. However, hardware-software contracts inherently assume a specific system context. When verification ignores this context, the results may not hold once the CPU is integrated into a platform. For example, the constant-time contract observer mode [15] exposes the program counter and all memory-access addresses, capturing timing effects of caches and of the memory hierarchy [15], [16]. Other modes, such as the architectural observer, reveal even more detail by also disclosing values exchanged with memory. Such observer modes depend on platform-level features (e.g., SoC components), yet many verification techniques overlook these modes or cannot flexibly capture platform-induced channels.

Platform timing contracts. We introduce platform timing contracts to extend hardware-software contracts to the broader system context in which a CPU operates. These contracts provide a language to express the expected timing behavior of the entire platform, including interactions with other components such as memory controllers, I/O devices, and caches. By considering these interactions, platform timing contracts provide a comprehensive understanding of potential timing channels and of the resulting security implications. To verify the confidentiality guarantees of a CPU integrated into a specific platform using existing techniques that verify a CPU in isolation, we introduce an automatic instrumentation that captures platform-specific timing information while retaining compatibility with all existing hardware-software contract verification techniques.

We apply the instrumentation to the 2-stage Sodor [23] and the 3-stage Kronos [24] RISC-V CPUs, which have been commonly used in constant-time verification benchmarks [19]–[21]. Not only does the verification time not increase significantly when applying the instrumentation if no violation is found, but we demonstrate the existence of platform-specific timing channels on typical platforms that would otherwise be missed by some existing verification techniques.

In summary, our contributions in this paper are:

- We introduce platform timing contracts that summarize platform-specific timing behaviors.
- We implement an open-source lightweight CPU instrumentation that accounts for all platform-induced timing side-channels for a given platform timing contract.
- We instrument the 2-stage Sodor and the 3-stage Kronos RISC-V CPUs. We show that the instrumentation incurs negligible verification performance overhead and effectively captures platform-specific timing channels that would be missed by some existing CPU constant-time verification techniques that operate in isolation.

Open sourcing. All our experiments can be found at: https://placeholder/

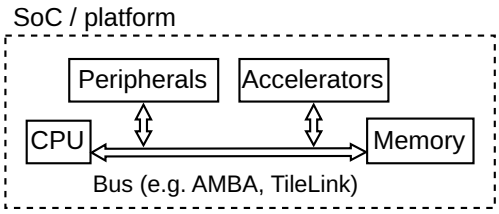

Fig. 1. Example SoC platform architecture.

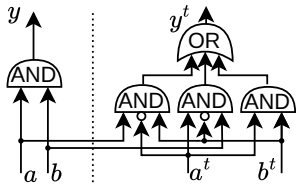

Fig. 2. Taint tracking at gate level [29]. Left: original circuit. Right: Added instrumentation for taint tracking. The original input signals are a and b. The taint bits corresponding to a and b are respectively a^t and b^t. The output taint bit y^t indicates whether the output is tainted, i.e., influenced by the tainted inputs.

II. Background

In this section, we provide background on SoC platforms, timing side-channels and non-interference properties.

A. SoC platforms

In this paper, we use the words SoC, system and platform interchangeably to refer to the integrated circuit that integrates various components, including CPUs, memory, and peripherals, as illustrated in Fig. 1. The communication between these components is typically managed through standardized bus protocols, such as AMBA [25] or TileLink [26]. The bounds of components are not always clear-cut. For example, Fig. 4 shows that caches can be considered part of the CPU or of the rest of the platform. For a hardware component, we refer to its *input signals* as the inbound wires, and its *inputs* as the values carried by those wires, and similarly for outputs.

B. Timing side-channels

Multiple hardware optimizations, such as caches and branch predictors, learn and exploit patterns in program execution to improve performance. In particular, the execution time of later instructions may depend on the outcome of earlier instructions. For example, accessing a cache line might be faster if the line has been brought into the cache recently by a prior memory access to a nearby location. Constant-time programming techniques, such as not using secret data as array indices or loop conditions, aim to ensure that secrets do not influence the execution time of a program through such channels [27], [28].

C. Information Flow Tracking

Hardware information flows capture properties such as confidentiality and integrity [30]. They are typically tracked using one of two techniques. The first technique is called dynamic information flow tracking (DIFT). It captures information flows by augmenting the hardware design under test (DUT) with a synthesizable instrumentation that computes the propagation of taint bits [19], [29], [31]–[33] as shown in Fig. 2. The second technique is called a miter circuit. It captures information flows by comparing the outputs of two copies of the DUT, where all inputs except the sources are unconstrained, while all the others are equal as illustrated in Fig. 3.

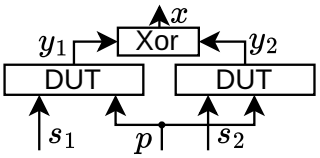

Fig. 3. Miter circuit. The two instances of the same DUT are supplied with the same public data p and unconstrained s_1 and s_2 secret data. The output bit y (i.e., instantiated for the instances in the miter as y_1 and y_2) reveals secret data if $x = 1$ is satisfiable.

D. Hardware-software contracts

Contract definition. Hardware-software contracts [15] split confidentiality responsibilities between the CPU and the software that it executes. For an initial microarchitectural state, a given program P, and the contents of memory containing public information M_p and secret information M_s, the contract specifies an execution mode and an observation mode. The *execution mode* determines which instructions are executed (potentially transiently) and generate events. The *observation mode* determines which of those events are exposed to an observer (possibly transiently), such as the addresses or values involved in memory accesses. Under such a contract, the CPU's behavior is summarized as a sequence of observable events known as a hardware trace [15], [16].

Contract verification. Multiple independent verification efforts have been conducted to ensure that hardware-software contracts are upheld during program execution, in particular regarding constant-time behavior [15], [17]–[22]. Most techniques consider a fixed contract [17]–[19], [21], while some others leverage custom, non-trivial technique-specific insights to verify a wider variety of contracts [20], [22] using the same technique.

Discussion. These techniques make assumptions about the platform in which the CPU is integrated, yet there exists no language for formally expressing and verifying them.

III. Platform Timing Contracts

In this section, we first provide a motivational example showing that the integration of a CPU into a system can introduce new timing channels that were not present in the CPU in isolation. We then discuss and formalize reentrant information flows. Finally, we introduce platform timing contracts to capture these reentrant flows.

A. Motivational example

Methodology (cache excluded). Fig. 4 (a) illustrates a small system that integrates a CPU, a cache and memory. We instantiate this system, using the Kronos [24] CPU as a case study, and a direct-mapped cache. We use the state-of-the-art

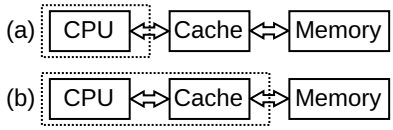

Fig. 4. Example verification scopes, represented as dotted boxes. (a) Cache excluded: the cache is considered part of the platform. (b) Cache included: the cache is considered part of the CPU.

VeloCT [18] tool to perform constant-time verification of the CPU, which is delimited by the dotted box. This tool verifies if only instructions of a given "safe instruction set" are executed, then the CPU runtime is independent of their operands. We consider memory instructions as part of the safe instruction set that should execute in constant-time, in addition to arithmetic RISC-V instructions such as `add`, `sub`, `xor`, etc.

Results (cache excluded). The Kronos verification with VeloCT succeeds. This indicates that in isolation, the CPU is constant-time secure. In particular, this result suggests that the timing of any program made of arithmetic and memory instructions does not depend on the data that is processed, even if this data is used as memory addresses.

Methodology (cache included). Fig. 4 (b) illustrates a scenario similar to Fig. 4 (a), but with the cache included in the verification scope, i.e., considered part of the CPU under verification. Except for this point, we use the same methodology as in the previous cache excluded case.

Results (cache included). The Kronos verification with VeloCT fails. This indicates that this time, the CPU is not constant-time secure. In particular, this result implies that the program execution timing can depend on secret information used as addresses of memory instructions.

Take-aways. While constant-time verification techniques typically operate at the level of a CPU in isolation (the entire platform specification might not yet be finalized at the CPU verification time), they may not account for all the potential interactions with other components in the system, such as caches or memory. Therefore, the integration of the CPU into a larger system can introduce new timing channels that were not present in isolation and that are hence not discovered by some existing constant-time verification techniques.

B. Reentrant flows

We now describe and formalize reentrant information flows, which are at the core of the timing channels that can be introduced by platform integration.

Netlist graphs. We describe a digital hardware system (e.g., the one illustrated in Fig. 4), as a netlist $G = (V, E)$, where V is the set of vertices representing logic gates such as AND, OR, NOT, flip-flops, or adders, and E is the set of directed wires between these vertices. We denote the subgraph of G that corresponds to the CPU that is integrated in this system as $G_C = (V_C, E_C)$, where $V_C \subseteq V$ and $E_C \subseteq E$. In particular, for all edges $(u, v) \in E_C$, both u and v are vertices in V_C. We denote the complementary subgraph as $G_P = (V_P, E_P)$, where $V_P = V \setminus V_C$ and $E_P = E \setminus E_C$, corresponding to the platform except the CPU that it integrates.

Constant-time programs and secret data. For a given constant-time verification technique, let us denote by $\mathcal{P}$ the set of constant-time programs that are considered for verification. The elements of the set $\mathcal{P}$ are triples $\tau = (P, M_p, M_s)$, where P is a constant-time program, M_p is the set of public memory valuations, and M_s is the set of secret values that the program may depend on. $\mathcal{P}$ depends on the specific constant-time verification technique being used. For example, for ConjunCT [17] and VeloCT [18], a triple (P, M_p, M_s) is considered constant-time (i.e., belongs to $\mathcal{P}$) if it contains no so-called "unsafe" instructions, i.e., instructions that can create an operand-dependent timing channel. For Tan et al. [21], a triple (P, M_p, M_s) is considered constant-time (i.e., belongs to $\mathcal{P}$) if it ensures that the program's control flow and all memory accesses are independent of secret data. The conditions for a program to be constant-time with respect to ConjunCT's and VeloCT's definition are stricter than those of Tan et al. [21], as the former generally forbid branches, for instance. Inclusively more restrictive constraints imply an inclusively smaller set $\mathcal{P}$ of constant-time programs.

Secret propagation. Constant-time verification techniques generally proceed by analyzing whether the valuation of a state element that captures timing differences can be influenced by secret information. For example, Tan et al. [21], LeaVe [20], ConjunCT [17] and VeloCT [18] monitor whether secret information can influence the commit signal, while μCFI [19] monitors whether secret information can influence a microarchitectural program counter structure.

We define the *valuation sequence* ν of a vertex. For a vertex $v \in V$ and a triple $\tau = (P, M_p, M_s)$, the valuation sequence $\nu_\tau(v)$ is the clock-accurate potentially infinite sequence of valuations that the vertex takes during the execution of a program P with respect to the memory valuations M_p and M_s, starting from the system's reset state, assumed deterministic. Note that we assume determinism for simplicity without loss of generality; to account for non-determinism, we could consider a set $\nu_\tau^{\text{non-det}}(v)$ of all possible valuation sequences instead of a single sequence $\nu_\tau(v)$, without change in the following reasoning.

We denote by V^s the set of vertices in V whose valuation sequence can be influenced by secret information when the system executes a program written in constant-time with respect to the secret data. An element $v \in V$ belongs to V^s if and only if there exist two triples $\tau = (P, M_p, M_s)$ and $\tau' = (P, M_p, M'_s)$ that only differ in M'_s, and such that $\tau \in \mathcal{P}$ and $\tau' \in \mathcal{P}$ but $\nu_\tau(v) \neq \nu_{\tau'}(v)$. We say that a vertex $v \in V$ is *secret-dependent* if it belongs to V^s.

For a secret-dependent vertex $v \in V^s$, we define the set P_v^s of *secret propagation paths* to be the paths in the graph G that connect the source of secrets (that can take the value M_s or M'_s) to v, where all the vertices in the path are secret-dependent. In particular, P_v^s is non-empty. Indeed, if $v \in V^s$, then at least one predecessor $u \in V$ of v must also be secret-dependent,

which recursively constructs a path in P_u^s.

Let us consider a strict subgraph $G' = (V', E') \subsetneq G$ and a vertex $v \in V'^s$. A path p in P_v^s is said to be *reentrant* in G' if it contains at least one vertex $u \in V \setminus V'$, i.e., a vertex that is outside of G'. Note that this u also belongs to $V^s \setminus V'$ because all nodes in p are secret-dependent by definition of P_v^s. If all paths in P_v^s are reentrant in G', then we say that the vertex v is *fully reentrant* in G'.

Example. Let us consider again the system Fig. 4 (a) where $\mathcal{P}$ describes programs where secret data (e.g., stored in a well-specified memory area) can be used as the address of a memory instruction. Because the CPU itself does not contain caches or other components with address-dependent timing, for v designating the commit signal (for LeaVe [20], Tan et al. [21], ConjunCT [17] or VeloCT [18]), or a microarchitectural PC signal (μCFI [19]), v is fully-reentrant in the CPU, for G describing the whole platform, and G' describing the CPU.

C. Contracts

The key idea of our work is to define *platform timing contracts* that capture the possible reentrant information flows in the CPU that are allowed by a given platform. Since the interface between the output signals of a CPU and the rest of a SoC is usually much simpler than the hardware-software interface, these contracts are expected to be simpler than existing hardware-software contracts.

Interface. We base our analysis on the widely-studied RISC-V ISA. The output signals of a RISC-V CPU are all through memory interfaces, which are made of data and control signals. Data signals transport the data to be read or written to or from the memory. Control signals transport the information about the memory transaction, such as the memory address, handshake signals, and depending on the memory bus protocol (e.g., AMBA AXI [25] or TileLink [26]), further control signals such as the type of transaction and the size of the data to be accessed. Input signals of a RISC-V CPU can be more diverse, including memory input data and control signals, interrupt lines, and various other control signals such as clock gating signals.

Platform timing contracts. Platform timing contracts aim to ensure that state elements that capture timing differences (e.g., a commit signal or a microarchitectural program counter) *are not fully reentrant* in the CPU. Hence, platform timing contracts specify the expected information flows between the output signals of the CPU and its input signals. We express a platform timing contract as a list of clauses, listed per CPU output signal.

Example contracts. Fig. 5 provides four example contracts for a RISC-V CPU with a Von Neumann architecture. We model the memory interface as an outbound address bus (`addr`), an inbound and an outbound memory data port (respectively `rdata` and `wdata`), and a simple valid-ready handshake protocol (inbound `resp` and outbound `req`), similar to the `req` and `gnt` handshake signals in the Hardware Processing Engines (HWPE) 2.0 interface protocol [34]. Fig. 5's contract C1 describes a simple platform with ideal memory that re-

(C1)	$\text{req} \rightarrow \Diamond\{\text{gnt}, \text{rdata}\}$ $\text{we} \rightarrow \Diamond\{\text{rdata}\}$ $\text{addr} \rightarrow \Diamond\{\text{rdata}\}$ $\text{wdata} \rightarrow \Diamond\{\text{rdata}\}$	$\text{req} \rightarrow \Diamond\{\text{gnt}, \text{rdata}, \text{int}\}$ $\text{we} \rightarrow \Diamond\{\text{gnt}, \text{rdata}, \text{int}\}$ $\text{addr} \rightarrow \Diamond\{\text{gnt}, \text{rdata}, \text{int}\}$ $\text{wdata} \rightarrow \Diamond\{\text{gnt}, \text{rdata}, \text{int}\}$	(C3)
(C2)	$\text{req} \rightarrow \Diamond\{\text{gnt}, \text{rdata}\}$ $\text{we} \rightarrow \Diamond\{\text{gnt}, \text{rdata}\}$ $\text{addr} \rightarrow \Diamond\{\text{gnt}, \text{rdata}\}$ $\text{wdata} \rightarrow \Diamond\{\text{rdata}\}$	$\text{req} \rightarrow \Diamond\{\text{gnt}, \text{rdata}, \text{int}\}$ $\text{we} \rightarrow \Diamond\{\text{gnt}, \text{rdata}, \text{int}\}$ $\text{addr} \rightarrow \Diamond\{\text{gnt}, \text{rdata}, \text{int}\}$ $\text{wdata} \rightarrow \Diamond\{\text{gnt},$ $\text{rdata if addr} \in \text{periph_range},$ $\text{int if addr} \in \text{periph_range}\}$	(C4)

Fig. 5. Example platform timing contracts. The syntax "output signal $\rightarrow \Diamond\{$input signals$\}$" indicates that variations in the valuation of this output signal can affect these input signals.

sponds with a constant timing (i.e., that is address-independent). A change in the memory transaction control signals from the CPU can only change the returned value. Only the request signal (`req`) can affect the existence, and hence the timing, of a memory response. The diamond $\Diamond$ underlines that the right-hand side of the implication can *eventually* happen, for any number of times after the left-hand side. Contract C2 adds timing dependence on the memory address, which is typical of caches. In this contract, `req` and `addr` have an identical clause. Indeed, a change in the address of a read, for example, can change a cache miss into a cache hit. This changes the timing of the response, and hence its value at some point in time, including the `gmt` control signal and the `rdata` data signals. Contract C3 models data-dependent interrupts (`int`) and timing. That is, the *reentrant secret propagation paths* (see Section III-B) all go through a specific set of memory addresses only. For example, some peripheral might be controlled through memory-mapped registers such as an interrupt controller [35].

Conditional refinements. Contract C3 implies that for a fixed address, changes in the data written to memory can eventually affect the existence of an interrupt or affect the timing of the memory response. For some platforms, this might only hold for specific address ranges where peripherals are mapped, while parts of the memory address range can be used in a data-independent timing, for example because it is mapped to memory. Hence, we introduce address-dependent contracts as in Contract C4, where the right-hand side of the implication depends on the memory address set `periph_range`, which is a fixed set of addresses. For example, this can be achieved with RISC-V Physical Memory Protection (PMP) in a setting where the most privileged execution mode does not access secret data and protects the `periph_range` address range from being accessed from lower privileges, in the philosophy of trusted execution environments [36]–[41].

Ordering of platform timing contracts. Like for hardware-software contracts [15], we can define a partial ordering of platform timing contracts. We say that a contract A is stronger than a contract B if for every output of the CPU (i.e., the left-hand side of the arrow in the contract), the set of possible CPU inputs tainted by this output in contract A is a subset of the set of possible CPU inputs tainted by this output in contract B. Said otherwise, contract A is stronger than contract B if it allows for fewer possible information flows. For example, Contract C4 and

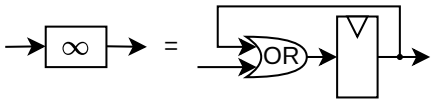

Fig. 6. Sticky-one operator. The sticky-one is reset to zero (not illustrated in the figure) and sticks to 1 once it is set.

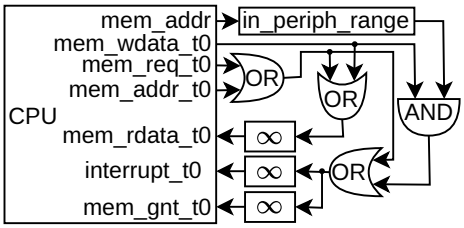

Fig. 7. Platform timing contract instrumentation for Contract C4 for a design whose information flows are tracked with dynamic information flow tracking instrumentation. Signals that end with `t0` represent the taint signals [29], [32].

Contract C2 are stronger than Contract C3. This creates a lattice structure like for hardware-software contracts [15]. A CPU that is verified to have a constant-time behavior with respect to a stronger platform timing contract is also verified with respect to a weaker platform timing contract.

IV. Instrumentation

In this section, we express these contracts as a synthesizable platform timing contract instrumentation (PTCI). Because it must materialize an information flow property, the PTCI cannot be directly applied to the CPU under verification. Indeed, in the original CPU before DIFT instrumentation or a miter construct, the notion of taint [19], [32] or two-trace property [17], [18], [20], [21] does not yet exist. Instead, the PTCI is applied to the construct that supports information flows, which is either a circuit with DIFT instrumentation, or a miter (see Section II-C).

Sticky-one operator. Fig. 6 introduces the sticky-one operator. It models the diamond $\Diamond$ operator in the instrumentations by making its output stick to 1 as soon as its input is 1. Its output returns to zero only when the system is reset.

PTCI for DIFT. For a CPU instrumented with DIFT [29], [32], we materialize contract information flows (i.e., arrows "$\rightarrow$" in a given contract) from a CPU output signal to a CPU input signal as a wire between the two corresponding taint signals, where a sticky-one operator accommodates information flows that might eventually occur (i.e., the $\Diamond$ operator). When multiple CPU outputs have a contract information flow to a single CPU input signal (i.e., there are several arrows "$\rightarrow$" in the contract whose right-hand set contains the same input signal), the corresponding taint signals are `OR`'ed together, meaning that the input signal can be influenced by any of these output signals. The PTCI for Contract C4 is shown in Fig. 7 for DIFT.

PTCI for miter constructs. The PTCI for a miter construct must ensure that the CPU input signals that are influenced by the CPU output signals are distinct between the two copies of the CPU. As shown in Fig. 8, this is achieved using `XOR` operations between the CPU outputs of the two copies. Ultimately, another `XOR` operation with the result of this difference is performed with the input signal, deciding whether for a given input signal, the two copies of the CPU receive the same input, or one copy receives a flipped input. Note that there is no conceptual difference between the PTCI for DIFT or for a miter, yet the former allows for a more compact PTCI.

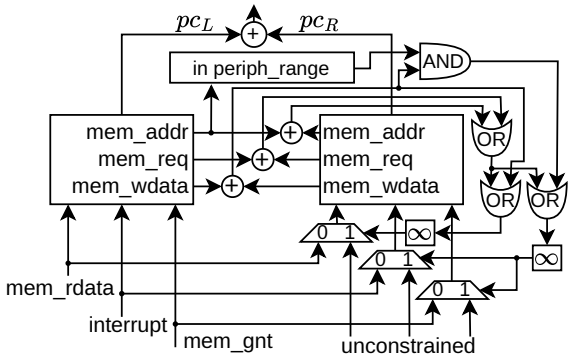

Fig. 8. Platform timing contract instrumentation for Contract C4 for a design whose information flows are tracked with a miter.

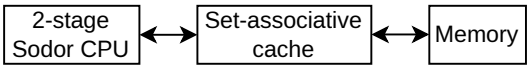

Fig. 9. Simple SoC architecture with a CPU, a cache, and memory.

V. Evaluation

In this section, we first present the PTCI implementation (Section V-A), evaluate its performance and the resulting soundness improvement (Section V-B), and finally verify the compliance of several platforms with the platform timing contracts discussed in this paper (Section V-C).

Evaluation setup. We execute the performance evaluations on a server equipped with an Intel Xeon E5-2667 CPU with 256 GB RAM. We use the 2-stage Sodor CPU [23] (commit `32d49f9`) and the 3-stage Kronos CPU [24] (commit `13678d4`), which are simple RISC-V CPUs already used in previous constant-time verification benchmarks [19]–[21].

A. PTCI implementation

The simple interface between the CPU and the platform makes the PTCIs simple, as earlier illustrated in Fig. 7 and Fig. 8. We implement a Yosys synthesizer pass [42] that instruments a CPU that is either instrumented with taint tracking logic such as GLIFT [29] or CellIFT [32], or part of a miter construct. The synthesizer pass takes as input a Verilog description of the CPU under test in one of these two configurations (instrumented for taint tracking or in a miter) and outputs a Verilog design instrumented with the PTCI. In addition to the CPU's Verilog description, the synthesizer pass takes a conditional contract as input, with a syntax similar to the one used in Fig. 5, and takes a machine-readable description of the CPU's interface ports. The instrumentation process takes a few seconds, including the boilerplate time taken by Yosys to process the designs.

TABLE I
VERIFICATION RESULTS FOR THE ORIGINAL (O) SODOR AND KRONOS, AND FOR PTCI-INSTRUMENTED (P) SODOR AND KRONOS FOR CONTRACT C3. RESULTS ARE "PASS" IF ALL MEMORY INSTRUCTIONS ARE CONSIDERED SAFE WITH REGARD TO ALL THEIR OPERANDS (IN PARTICULAR WITH REGARD TO THE ADDRESS). TIME INDICATES THE TIME TO OBTAIN A PROOF FOR A SUBSET OF INSTRUCTIONS THAT EXCLUDES MEMORY INSTRUCTIONS IF THEY ARE SHOWN TO BE NON-CONSTANT-TIME. THE TIME SHOWN FOR μCFI IS THE MEAN ACROSS INSTRUCTIONS.

	μCFI		**VeloCT**	
	Sodor	**Kronos**	**Sodor**	**Kronos**
Time (O)	8 min	31 min	1 min	1 min
Result (O)	Pass ✓	Pass ✓	Pass ✓	Pass ✓
Time (P)	8 min	31 min	1 min	1 min
Result (P)	Fail ✗	Fail ✗	Fail ✗	Fail ✗

TABLE II
VERIFICATION OF THE COMPLIANCE OF PLATFORMS TO PTCI.

Contract	`nocache`	`cache`	`interrupt`
C1	Pass ✓	Fail ✗	Fail ✗
C2	Pass ✓	Pass ✓	Fail ✗
C3	Pass ✓	Pass ✓	Pass ✓
C4-a	Pass ✓	Pass ✓	Pass ✓
C4-b	Pass ✓	Pass ✓	Fail ✗

B. Soundness improvement with PTCI

We have shown in Section III-A that the security guarantees provided by VeloCT, and similarly with μCFI, may not hold when the CPU is integrated into a platform that contains elements with address-dependent timing such as caches. We first verify Sodor and Kronos with μCFI (DIFT-based) and VeloCT (miter-based). ConjunCT is not open-source but its authors affirmed that it is superseded by VeloCT. We then instrument Sodor and Kronos with the PTCI and verify them again. We report the verification results, as well as the time taken by each tool to formulate a proof in Table I. The difference in verification time with and without the PTCI is not significant. We observe that the PTCI indeed can correct the unsoundness of μCFI and VeloCT with regard to platform integration such as cache side channels.

C. Platform verification

Finally, we verify the compliance of platforms with the platform timing contract. Because these contracts specify a form of non-interference, they can be verified conveniently with miter constructs, similar to how most techniques verify constant-time properties for CPUs [17], [18], [20], [21]. We verify three platforms. The `cache` platform is a simple SoC architecture composed of a CPU, a set-associative cache and memory, as depicted in Fig. 9. The `nocache` platform is like the `cache` platform, but the cache is removed, connecting the CPU directly to the memory, which is modeled as having no address-dependent latency. The `interrupt` platform is like the `cache` platform, but also adds an uncached interrupt controller at the address `0x1000`, which generates an interrupt signal when a non-zero value is written to this address. For each of these platforms, we verify Contracts C1-C4 with `periph_range` as `0x1000` to `0x1FFF` (C4-a), or `periph_range` as `0x2000` to `0x2FFF` (C4-b). We construct miters whose DUT is the platform seen from the CPU's perspective, i.e., whose input signals are the CPU output signals, and whose output signals are the CPU input signals. Note that the CPU itself is not part of this miter. To verify, for example, the last arrow of Contract C4, we enforce all inputs of the miter to be identical between the two sides of the miter except for the `mem_wdata` input, and we additionally constrain the `mem_addr` input, identical between the two sides of the miter, not to be in `periph_range`. We then verify that only `mem_rdata` can be affected by changes to `mem_wdata` by `XOR`'ing the outputs of `mem_gnt` for the two copies of the platform in the miter together and expressing a SAT formula on this signal, which should not be satisfiable. Table II summarizes the verification results obtained with Cadence Jaspergold for the different platforms. The total duration of the experiment does not exceed one minute.

VI. DISCUSSION

ISAs like RISC-V communicate with the rest of the system through memory interfaces. Some other ISAs like x86 have special I/O instructions that are not memory-mapped. The reasoning about platform timing contracts for these ISAs typically incurs more clauses (i.e., lines with arrows $\rightarrow$) in the contracts to account for these two separate channels for communicating with the system. System-level instructions of ISAs like MIPS [43] (e.g., the `mfc0` instruction) or ARM (e.g., the `SVC` instruction) also require additional clauses.

VII. RELATED WORK

Previous work introduced mitigations against cache timing attacks [44]–[48] or network-on-chip interference [49]–[54]. Our work is orthogonal to these works and models a platform where some channels are tolerated (e.g., cache timing). Some hardware-software contracts and constant-time techniques [15], [20], [21] typically assume that addresses of memory operations might eventually influence timing. Our instrumentation is also compatible with techniques that require more manual effort [55]. In conclusion, the PTCI that we introduce proposes fine-grained platform integration contracts, and makes them synthesizable so that they can directly be used upon techniques that are unaware of reentrant information flows.

VIII. CONCLUSION

We introduced platform timing contracts as an extension of hardware-software contracts. They provide a language for specifying expected timing channels in full-system contexts rather than isolated CPUs. To make such reasoning compatible with existing CPU timing side-channel verification frameworks, we proposed an instrumentation that accounts for platform-specific timing effects. Our evaluation shows a negligible verification performance overhead while uncovering timing channels that would otherwise remain invisible in isolation.

REFERENCES

[1] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher *et al.*, "Spectre attacks: Exploiting speculative execution," in *IEEE SP*, 2019.

[2] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, "Meltdown: Reading kernel memory from user space," in *USENIX Security*, 2018.

[3] S. Van Schaik, A. Milburn, S. Österlund, P. Frigo, G. Maisuradze, K. Razavi, H. Bos, and C. Giuffrida, "Ridl: Rogue in-flight data load," in *IEEE SP*, 2019.

[4] H. Ragab, E. Barberis, H. Bos, and C. Giuffrida, "Rage against the machine clear: A systematic analysis of machine clears and their implications for transient execution attacks," in *USENIX Security*, 2021.

[5] J. Wikner and K. Razavi, "Retbleed: Arbitrary speculative code execution with return instructions," in *USENIX Security*, 2022.

[6] S. Ruegge, J. Wikner, and K. Razavi, "Branch privilege injection: Compromising spectre v2 hardware mitigations by exploiting branch predictor race conditions," in *USENIX Security*, 2025.

[7] D. J. Bernstein, "Cache-timing attacks on aes," 2005.

[8] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee, "Last-level cache side-channel attacks are practical," in *IEEE SP*, 2015.

[9] Y. Yarom and K. Falkner, "FLUSH+RELOAD: A high resolution, low noise, l3 cache Side-Channel attack," in *USENIX Security*, 2014.

[10] Y. Yarom, D. Genkin, and N. Heninger, "Cachebleed: A timing attack on openssl constant time RSA," in *CHES 2016*. Springer, 2016.

[11] A. Moghimi, T. Eisenbarth, and B. Sunar, "Memjam: A false dependency attack against constant-time crypto implementations in SGX," in *Topics in Cryptology*. Springer, 2018.

[12] D. Gruss, C. Maurice, A. Fogh, M. Lipp, and S. Mangard, "Prefetch side-channel attacks: Bypassing SMAP and kernel ASLR," in *ACM CCS*, 2016.

[13] S. Van Schaik, A. Milburn, S. Österlund, P. Frigo, G. Maisuradze, K. Razavi, H. Bos, and C. Giuffrida, "Addendum to ridl: Rogue in-flight data load," 2019, https://mdsattacks.com/files/ridl-addendum.pdf.

[14] B. Herzog, S. Reif, J. Preis, W. Schröder-Preikschat, and T. Hönig, "The price of meltdown and spectre: Energy overhead of mitigations at operating system level," in *EuroSec*, 2021.

[15] M. Guarnieri, B. Köpf, J. Reineke, and P. Vila, "Hardware-software contracts for secure speculation," in *IEEE SP*, 2021.

[16] O. Oleksenko, C. Fetzer, B. Köpf, and M. Silberstein, "Revizor: Testing black-box cpus against speculation contracts," in *ASPLOS*, 2022.

[17] S. Dinesh, M. Parthasarathy, and C. W. Fletcher, "Conjunct: Learning inductive invariants to prove unbounded instruction safety against microarchitectural timing attacks," in *IEEE SP*, 2024.

[18] S. Dinesh, Y. Zhu, and C. W. Fletcher, "H-houdini: Scalable invariant learning," in *ASPLOS*, 2025.

[19] K. Ceesay-Seitz, F. Solt, and K. Razavi, "μcfi: Formal verification of microarchitectural control-flow integrity," in *CCS*, 2024.

[20] Z. Wang, G. Mohr, K. von Gleissenthall, J. Reineke, and M. Guarnieri, "Specification and verification of side-channel security for open-source processors via leakage contracts," in *CCS*, 2023.

[21] Q. Tan, Y. Yang, T. Bourgeat, S. Malik, and M. Yan, "Rtl verification for secure speculation using contract shadow logic," *ASPLOS*, 2025.

[22] Y. Hsiao, N. Nikoleris, A. Khyzha, D. P. Mulligan, G. Petri, C. W. Fletcher, and C. Trippel, "Rtl2mμpath: Multi-μpath synthesis with applications to hardware security verification," in *MICRO*, 2024.

[23] B. A. Research, "riscv-sodor," https://github.com/ucb-bar/riscv-sodor, [Online; accessed 8-April-2025].

[24] Kronos RISC-v (cascade fixes integrated). Accessed: 2024-07-10. [Online]. Available: https://github.com/SonalPinto/kronos

[25] "Amba axi and ace protocol specification," Arm Ltd., Tech. Rep. ARM IHI 0022H, 2017. [Online]. Available: https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/IHI0022H_amba_axi_protocol_spec.pdf

[26] CHIPS Alliance, *TileLink Specification*, 2020, available online. [Online]. Available: https://chipalliance.org/specs/tilelink/tilelink-spec-1.8.1.html

[27] D. A. Osvik, A. Shamir, and E. Tromer, "Cache attacks and countermeasures: The case of AES," in *Topics in Cryptology*. Springer, 2006.

[28] J. B. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, and M. Emmi, "Verifying constant-time implementations," in *USENIX Security*, 2016.

[29] M. Tiwari, H. M. Wassel, B. Mazloom, S. Mysore, F. T. Chong, and T. Sherwood, "Complete information flow tracking from the gates up," in *ASPLOS*, 2009.

[30] W. Hu, A. Ardeshiricham, and R. Kastner, "Hardware information flow tracking," *ACM CSUR*, 2021.

[31] A. Ardeshiricham, W. Hu, J. Marxen, and R. Kastner, "Register transfer level information flow tracking for provably secure hardware design," in *DATE*, 2017.

[32] F. Solt, B. Gras, and K. Razavi, "Cellift: Leveraging cells for scalable and precise dynamic information flow tracking in rtl," in *USENIX Security*, 2022.

[33] F. Solt and K. Razavi, "Hybridift: Scalable memory-aware dynamic information flow tracking for hardware," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024.

[34] PULP Platform Documentation, *HWPE-Mem Protocol*, PULP Platform, 2024, online; accessed July 4, 2025. [Online]. Available: https://hwpe-doc.readthedocs.io/en/latest/protocols.html

[35] RISC-V International Task Group, "Risc-v platform-level interrupt controller specification," RISC-V International, Technical Report Version 1.0.0 (ratified March 11, 2023), 2023. [Online]. Available: https://github.com/riscv/riscv-plic-spec/blob/master/riscv-plic-1.0.0.pdf

[36] D. Lee, D. Kohlbrenner, S. Shinde, K. Asanović, and D. Song, "Keystone: An open framework for architecting trusted execution environments," in *EuroSys*, 2020.

[37] V. Costan, I. Lebedev, and S. Devadas, "Sanctum: Minimal hardware extensions for strong software isolation," in *USENIX Security*, 2016.

[38] ARM Ltd., "Arm security technology—building a secure system using trustzone technology," ARM Ltd., Whitepaper, 2009. [Online]. Available: https://developer.arm.com/documentation/den0028/a/

[39] B. McGillion, T. Dettenborn, T. Nyman, and N. Asokan, "Opentee: An open virtual trusted execution environment," *arXiv preprint arXiv:1506.07367*, 2015.

[40] T. Bourgeat, I. Lebedev, A. Wright, S. Zhang, Arvind, and S. Devadas, "Mi6: Secure enclaves in a speculative out-of-order processor," in *arXiv preprint arXiv:1812.09822*, 2018.

[41] F. Brasser, P. Jauernig, F. Pustelnik, A.-R. Sadeghi, and E. Stapf, "Trusted container extensions for container-based confidential computing," in *arXiv preprint arXiv:2205.05747*, 2022.

[42] C. Wolf, J. Glaser, and J. Kepler, "Yosys-a free verilog synthesis suite," in *Austrochip*, 2013.

[43] C. Price, *MIPS IV Instruction Set (Revision 3.2)*, MIPS Technologies, Inc., Mountain View, CA, USA, Sep. 1995.

[44] A. Kar, X. Liu, Y. Kim, G. Saileshwar, H. Kim, and T. Krishna, "Mitigating timing-based noc side-channel attacks with llc remapping," *IEEE Computer Architecture Letters*, vol. 22, no. 1, 2023.

[45] G. Saileshwar and M. Qureshi, "Mirage: Mitigating conflict-based cache attacks with a practical fully-associative design," in *USENIX Security*, 2021.

[46] L. Giner, S. Steinegger, A. Purnal, M. Eichlseder, T. Unterluggauer, S. Mangard, and D. Gruss, "Scatter and split securely: Defeating cache contention and occupancy attacks," in *IEEE SP*, 2023.

[47] M. Werner, T. Unterluggauer, L. Giner, M. Schwarz, D. Gruss, and S. Mangard, "Scattercache: thwarting cache attacks via cache set randomization," in *USENIX Security*, 2019.

[48] M. K. Qureshi, "Ceaser: Mitigating conflict-based cache attacks via encrypted-address and remapping," in *IEEE Micro*, 2018.

[49] H. M. Wassel, Y. Gao, J. K. Oberg, T. Huffmire, R. Kastner, F. T. Chong, and T. Sherwood, "Surfnoc: A low latency and provably non-interfering approach to secure networks-on-chip," *ACM SIGARCH Computer Architecture News*, 2013.

[50] Y. Wang and G. E. Suh, "Efficient timing channel protection for on-chip networks," in *IEEE/ACM NOCS*, 2012.

[51] M. Schoeberl, F. Brandner, J. Sparsø, and E. Kasapaki, "A statically scheduled time-division-multiplexed network-on-chip for real-time systems," in *IEEE/ACM NOCS*, 2012.

[52] A. Psarras, J. Lee, I. Seitanidis, C. Nicopoulos, and G. Dimitrakopoulos, "Phasenoc: Versatile network traffic isolation through tdm-scheduled virtual channels," *IEEE TCAD*, 2015.

[53] M. G. Alonso, J. Flich, M. Turki, and D. Bertozzi, "A low-latency and flexible tdm noc for strong isolation in security-critical systems," in *IEEE MCSoC*. IEEE, 2019.

[54] M. S. Sadeghi, S. B. Sarmadi, and S. Hessabi, "Toward on-chip network security using runtime isolation mapping," *ACM TACO*, 2019.

[55] L. Deutschmann, A. Meza, D. Stoffel, W. Kunz, and R. Kastner, "Fastpath: A hybrid approach for efficient hardware security verification," in *DAC*, 2025.